

Runtime Analysis of Hybrid Sort

Gradient Descent Methodology

D. Tan, H. G. De Vera, P. Francis

Gradient Descent Methodology

Runtime Model

$$F(n, s, \theta) \approx ans + bn \log n - bn \log s, \text{ where } \theta = \begin{bmatrix} a \\ b \end{bmatrix}$$

- We wish to fit the model above to our data on runtime.
- Our runtime data y consists of (n, s, t_{time}) tuples.

Goal: Minimize Squared Loss

$$C(y, \theta) = \frac{1}{|y|} \sum_{i=1}^{|y|} [t_{\text{time},i} - F(n_i, s_i, \theta)]^2$$

- We model this regression problem as a squared-loss minimization problem. Given fixed y , we wish to find the value of θ that minimizes $C(y, \theta)$.
- We use the **Tensorflow** library to perform **gradient descent** along parameter space.

Ideal Initialization Property: Vector has Expected Norm 1

$$\mathbb{E}[|\theta|] = 1$$

- We initialize all weights with the same normal distribution with mean 0— $\mathcal{N}(0, \sigma^2)$.
- Let $X \sim \mathcal{N}(0, \sigma^2)$. Then, notice that:

$$\mathbb{E}[X^2] = \text{Var}(X) + \mathbb{E}[X]^2 = \sigma^2 + 0^2 = \sigma^2$$

- Hence, we have:

$$|\mathbb{E}[\theta]| = \mathbb{E}\left[\sqrt{|\theta|X^2}\right] = \sqrt{|\theta|\mathbb{E}[X^2]} = \sigma\sqrt{|\theta|} = 1$$

$$\sigma = \frac{1}{\sqrt{|\theta|}} = \frac{1}{\sqrt{2}}$$

- Directly running gradient descent on our original model leads to **exploding gradients**. This is most likely caused by an unconstrained output space.
- Hence, taking inspiration from Artificial Neural Networks, we introduced an **activation function** σ :

$$\sigma(x) := \frac{10\sqrt{10}\sqrt[3]{x}}{1 + 10\sqrt{10}|\sqrt[3]{x}|}$$

- Intuitively, σ clamps unconstrained real inputs to $[-1, 1]$. This mitigates large output values, which may lead to exploding gradients during backpropagation.
- We also applied **gradient clipping** to prevent uncontrolled explosions.

- With this activation function, we redefined our cost function as follows:

Goal: Minimize Squared Loss

$$C_{\text{norm}}(y, \theta) = \frac{1}{|y|} \sum_{i=1}^{|y|} \left[\sigma(t_{\text{time},i}) - \sigma(F(n_i, s_i, \theta)) \right]^2$$

- Intuitively, this **smoothens** the cost surface, facilitating gradient descent.
- This smoothing transformation, however, removes the resolution required to hone in on very accurate parameter values θ .

- **Insight:** Use C_{norm} to capture the **high-level geometry** of the cost surface. Use C to capture the **details** of the cost surface.
- This insight motivates the following training procedure:
 - **Find an Approximate Soln:** Run gradient descent for 1000 epochs with C_{norm} .
 - **Refine the Approximation:** Run gradient descent for 1000 epochs with C .
- **Analogy:** To find the highest point in a mountain range, first look for the correct mountain, then find the highest point on that mountain.

- Another observation is that the optimal a, b satisfy $a \geq 0$ and $b \geq 0$.
- The model is **unaware** of this fact and may search regions where a or b are negative.
- We believed simply clipping a and b to nonnegative values would be too destructive, potentially leading to gradient descent getting stuck around the boundaries of the admissible parameter space.
- **Insight:** Define two new parameters a' and b' satisfying $a'^2 = a, b'^2 = b$.
- Hence, we **implicitly clip** the parameters by exploiting the nonnegativity of $y = x^2$.

Updated Runtime Model

$$F(n, s, \theta') \approx a'^2 ns + b'^2 n \log n - b'^2 n \log s, \text{ where } \theta' = \begin{bmatrix} a' \\ b' \end{bmatrix}$$

Runtime Model

$$F(n, s, \theta') \approx a'^2 ns + b'^2 n \log n - b'^2 n \log s, \text{ where } \theta' = \begin{bmatrix} a' \\ b' \end{bmatrix}$$

Cost Functions

$$C(y, \theta) = \frac{1}{|y|} \sum_{i=1}^{|y|} [t_{\text{time},i} - F(n_i, s_i, \theta)]^2$$

$$C_{\text{norm}}(y, \theta) = \frac{1}{|y|} \sum_{i=1}^{|y|} [\sigma(t_{\text{time},i}) - \sigma(F(n_i, s_i, \theta))]^2$$

- **Goal:** Find $\theta' \in \mathbb{R}^2$ that minimizes $C(y, \theta')$.
- **The Algorithm:**
 1. Run Gradient Descent under the cost function $C_{\text{norm}}(y, \theta')$, with the initial weights sampled from $\mathcal{N}\left(0, \frac{1}{\sqrt{2}}\right)$.
 2. Run Gradient Descent under the cost function $C(y, \theta')$, with the initial weights set to the final weight values from the first step.

- This algorithm converges with $R^2 = 0.9996$.
- An implementation of this algorithm may be found in `model_regression_analysis.py`.
- We acknowledge the existence of exact analytical methods to minimize our objective function, which is linear-in-form.
- We have opted to use Tensorflow as a means of exploration (and for fun)!